

[← Go Back](#)

Hardware

Nano Matter

Tutorials

[Nano Matter User Manual](#)[Getting Started with Nano Matter Display](#)[Matter Smart Fan with the Arduino Nano Matter](#)[Matter Smart Relay with the Arduino Nano Matter](#)[Matter RGB Light with the Arduino Nano Matter](#)[Matter Temperature Sensor with the Arduino Nano Matter](#)[Open Thread Border Router with Nano Matter & ESP32](#)[ML Magic Wand with the Arduino Nano Matter](#)[Home](#) / [Hardware](#) / [Nano Matter](#) / **ML**

Magic Wand with the Arduino Nano Matter

ML Magic Wand with the Arduino Nano Matter

Learn how to build a Gesture Recognition Magic Wand using the Arduino Nano Matter

Author Christopher Méndez, Leonardo Cavagnis and Andrea Richetta · Last revision 05/22/2025

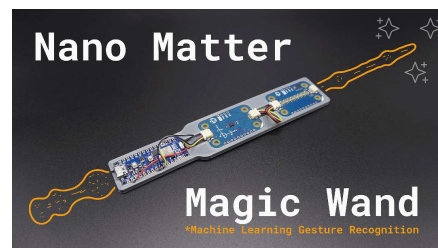
ON THIS PAGE

Overview

[Goals](#)[Hardware and Software](#)[Requirements](#)[How to Magic Wand Works?](#)[Project Setup](#)[Project Testing](#)[Conclusion](#)

Overview

This tutorial describes how to build a gesture recognition system based on a machine learning model using TensorFlow Lite and the Arduino Nano Matter.



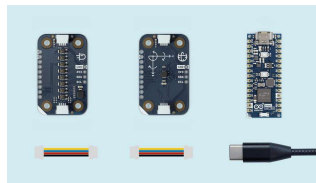
The Arduino Nano Matter acts as a digital magic wand ✨, where sensor data from its movements is processed by a model to classify and detect specific gestures. The inference results will turn on Modulino Pixels on a specific color respectively to the detected gesture.

Goals

[Help](#)

The goal of this project tutorial is to showcase the capabilities of the Arduino Nano Matter running Tiny Machine Learning models on the edge for gesture recognition. The wand can detect two gestures drawn in the air: **"W" (wing gesture)** and **"O" (ring gesture)**.

Hardware and Software Requirements



Hardware Required

Hardware Requirements

[Arduino Nano Matter](#) (x1)

Modulino Movement (x1)

Modulino Pixels (x1)

Qwiic cables (x2)

[USB-C® cable](#) (x1)

[Custom 3D printed parts](#)

Software Requirements

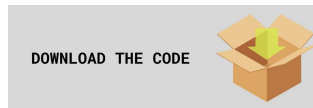
[Arduino IDE 2.0+](#) or [Arduino Cloud Editor](#)

[Modulino library](#). This library adds the support for the Modulino nodes, you can install it from the **Library Manager** in the Arduino IDE.

[Silicon Labs core](#). This enables the Silicon Labs hardware including the Arduino Nano

it from the **Boards Manager** in the Arduino IDE.

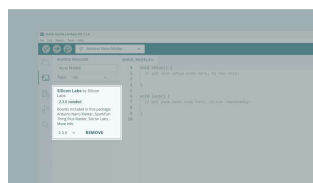
Download the Project Code



Download the complete project code [here](#).

Board Core and Libraries

The **Silicon Labs** core contains the libraries and examples you need to work with the board's components, such as its Matter, Bluetooth® Low Energy, and I/Os. To install the Nano Matter core, navigate to **Tools > Board > Boards Manager** or click the Boards Manager icon in the left tab of the IDE. In the Boards Manager tab, search for `Nano Matter` and install the latest `Silicon Labs` core version.



Installing the Silicon Labs core in the Arduino IDE



The core version for this tutorial must be 2.3.0 or greater for the support of the Silabs Tensorflow Lite library.

How to Magic Wand Works?

In its **idle state**, the wand's LEDs remain **solid blue**, indicating that it is ready to detect a gesture.

When the wand is moved, the LEDs **turn off**, showing that gesture recognition has started.



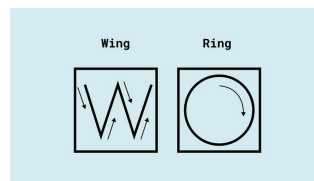
Gesture Colors

Once the recognition process is complete:

If a gesture is successfully detected, the LEDs **blink rapidly for 4 seconds** in the color assigned to the recognized gesture:

Green for "W" (wing gesture)

Yellow for "O" (ring gesture)



Recognizable Gestures

After this, the wand returns to its **idle state**, with the LEDs **solid blue** again.

Gestures should be performed with wide and slow movements,

i lasting approximately 1-2 seconds for optimal recognition.

Data Collection

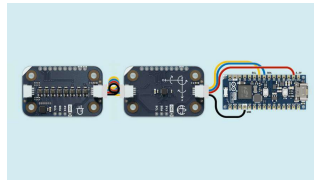
Accelerometer sensor readings are collected at a frequency of **104 Hz**, with a sample duration of

results in **200** tuples of accelerometer (aX, aY, aZ) sensor data for each sample ($200 * 1/104 \text{ Hz} = 1.92\text{s}$). Therefore, a total of **600 data points** are collected for each gesture ($200 * 3 = 600$).

Project Setup

Schematic Diagram

Use the following connection diagram for the project:



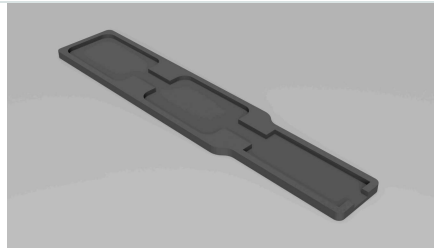
Project Wiring Diagram

The Modulino are daisy-chained leveraging the Qwiic I2C connection with the Nano Matter. You can use the Nano Connector Carrier that includes a Qwiic connector, or you can directly solder the wires to the Nano Matter following the mapping below:

Qwiic Connection	Color	Arduino Pin
SCL	Yellow	A5
SDA	Blue	A4
VCC	Red	3.3V
GND	Black	GND

3D Printed Parts

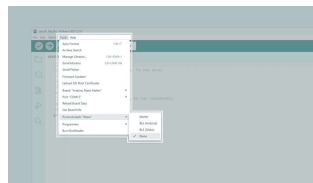
You can mount the Nano Matter and the Modulino nodes on a custom 3D



Programming

Let's go through some important code sections to make this project fully operational, starting with the Protocol Stack setting and required libraries:

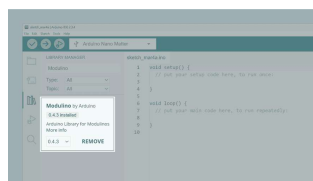
In the Arduino IDE upper menu, after selecting the **Nano Matter board** from the Silicon Labs core, navigate to **Tools > Protocol stack** and select **None**.



Protocol stack configuration

i The code will only compile if the **Protocol Stack** is set to **None**.

Download the [Modulino](#) library from the Arduino IDE Library Manager. This will enable the support for the Modulino Pixels and the Modulino Movement.



Modulino Library Installation



The Modulino library for this tutorial must be 0.4.3 or greater.

You can download the **complete project code** from [here](#) or copy and paste it from the snippet below:

```

1  #include "SilabsTFLite/
2  #include "Modulino.h"
3
4  #define SEQUENCE_LENGTH
5  #define SIGNAL_CHANNEL
6
7  #define GESTURE_COUNT
8  #define WING_GESTURE
9  #define RING_GESTURE
10 #define NO_GESTURE
11
12 #define DETECTION_THRE
13
14 static TfliteTensor* mc
15 static tflite::MicroInt
16 static int input_length
17 static TfliteTensor *ou
18
19 typedef struct model_out
20     float gesture[GESTURE
21 } model_output_t;
22
23 typedef float acc_data_
24
25 ModulinoColor OFF(0, 0,
26 ModulinoColor YELLOW(25
27
28 ModulinoMovement imu;

```



Be aware that your sketch needs the model header `sl_tflite_micro_model.c` that is included in the [complete code download file](#).

After the libraries import, several variables and structures are declared that has to be with the Machine Learning implementation:

```

1  #include "SilabsTFLiteMic
2  #include "Modulino.h"
3
4  #define SEQUENCE_LENGTH
5  #define SIGNAL_CHANNELS
6
7  #define GESTURE_COUNT
8  #define WING_GESTURE
9  #define RING_GESTURE
10 #define NO_GESTURE
11
12 #define DETECTION_THRESHO
13
14 static TfliteTensor* mode
15 static tflite::MicroInter
16 static int input_length;
17 static TfliteTensor *outp
18
19 typedef struct model_outp
20     float gesture[GESTURE_C
21 } model_output_t;
22
23 typedef float acc_data_t;

```

We define some function prototypes to manage the Modulino Pixels and the Modulino Movement accelerometer data. Also, the classes for the Modulino Nodes are declared.

```

1  ModulinoColor OFF(0, 0, 0
2  ModulinoColor YELLOW(255,
3
4  ModulinoMovement imu;
5  ModulinoPixels leds;
6
7  void setPixel(int pixel,
8     leds.set(pixel, color,
9     leds.show());
10 }
11
12 bool accelerometer_setup(
13 void accelerometer_read(a

```

In the `setup()` function, the **Tensorflow Lite model** is initialized alongside some modules support

Serial communication

Modulino Pixels LEDs

Modulino Movement IMU

At the beginning, relevant model parameters as the input tensor and its characteristics are printed in the Serial Monitor for debugging.

```

1  void setup() {
2    Serial.begin(115200);
3    Serial.println("Magic");
4    Serial.println("init...");
5
6    // Init TFLite model
7    sl_tflite_micro_init()
8
9    // Obtain pointer to model input tensor
10   model_input = sl_tflite_micro_get_input(0);
11   interpreter = sl_tflite_micro_get_interpreter();
12   output = sl_tflite_micro_get_output(0);
13
14
15   // Check model input parameters
16   if ((model_input->dims[0] != 1) ||
17       || (model_input->dims[1] != 1) ||
18       || (model_input->dims[2] != 1) ||
19       || (model_input->dims[3] != 1)) {
20     Serial.println("error: invalid model input parameters");
21     while(1) ;
22   }
23
24   // Initialize accelerometer
25   bool setup_status = accelerometer_init();
26   if (!setup_status) {
27     Serial.println("error: failed to initialize accelerometer");
28     while(1) ;
29   }

```

In the `loop()` function, a four steps process is executed as following:

- 1 **Significant movement detection:** The Nano Matter reads the accelerometer data searching for significant movement, if detected then the next step starts.
- 2 **Accelerometer values**

accelerometer values and normalized in the model format.

- 3 **Inference run:** The inference is run, and its results are retrieved for visual feedback triggering.
- 4 **Feedback results:** The inference results are analysed, printed in the Serial Monitor and reflected on the Modulino Pixels.

```

1 void loop() {
2   acc_data_t *dst = (ac
3
4   // Wait until a signi
5   bool movementDetected
6   Serial.println("Waiti
7
8   while (!movementDetec
9     uint8_t acceleroSta
10    acceleroStatus = in
11
12    // Proceed only if
13    if (acceleroStatus
14      float acceleratic
15      imu.update();
16
17      acceleration[0] =
18      acceleration[1] =
19      acceleration[2] =
20
21      // Calculate the
22      float absSum = fa
23
24      // If the movemer
25      if (absSum > 1.8)
26        movementDetec
27        Serial.printl
28    }

```

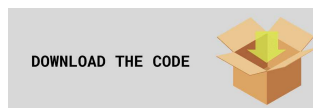
In addition, some helper functions are used:

`accelerometer_setup()`: This function initialize the Modulino Movement accelerometer and returns true if it did it successfully.

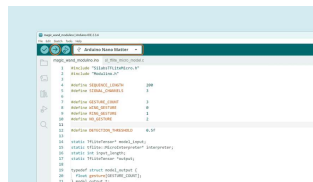
accelerometer data, normalize it and prepare the model input buffer.

Upload the Project Sketch

You can download the code from [here](#) or by clicking on the image below:



In the Arduino IDE select the **Arduino Nano Matter** inside the *Silicon Labs* board package and make sure the **Protocol Stack** is set to *None*.

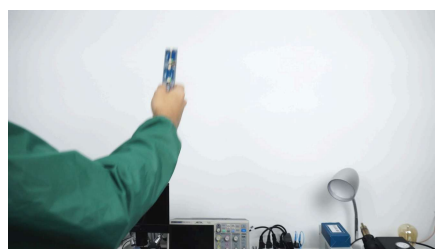


Nano Matter Sketch
Upload

After the code is uploaded successfully, you can test your Nano Matter Magic Wand.

Project Testing

Power the Nano Matter of your Magic Wand using a USB cable and start moving it following the **Wing** or **Ring** gesture to see if it recognizes it.



In this tutorial you learned how to use the Arduino Nano Matter as a Machine Learning gesture recognizer; leveraging the optimized Silicon Labs library for Tiny Machine Learning, we showcased the board capabilities of running TinyML algorithms with ease thanks to the **hardware acceleration** achieved with the **Matrix Vector Processor (MVP)** integrated into the Nano Matter microcontroller.

Next Steps

Extend your knowledge about the Nano Matter following the [Arduino Documentation](#).

Improve gesture recognition model accuracy by incorporating gyroscope data.

Create your own custom gestures recognition model.

Create machine learning models leveraging other Modulino sensors.

Suggest changes	Need support?	License
The content on docs.arduino.cc is facilitated through a public GitHub repository . If you see anything wrong, you can edit this page here .	Help Center Ask the Arduino Forum Discover Arduino Discord	The Arduino documentation is licensed under the Creative Commons Attribution-Share Alike 4.0 license.

Was this article helpful?

